

AI Concierge — Conversational Context v1 (Planning Spec)

Severity classification: **Level 3 — Split plan before implementation.**

Reason: durable design touches router/intent classification, request/response contract, session state, frontend chat shell, observability, and tests. Single-PR implementation would violate the implementation skill's 3-category rule. Spec only here, per request.

1. Working assumptions

- The backend already has a typed two-stage router (`app/concierge/router.py`) returning `place_recommendations | trip_advice | unsupported`. We add a third orthogonal axis ("turn intent": new search vs. follow-up on previous cards) without rewriting that router.
- Verified card identity is already stable: cards carry `provider_place_id`, `google_maps_uri`, `google_verification.business_status="OPERATIONAL"`. These are the trust gates we must not weaken.
- Persisted chat already exists: `concierge_messages` (migration 002), and assistant rows already store the full structured response in `structured_results` JSONB. The previous verified cards are already on disk per trip — v1 reuses that, so we add no new table.
- The frontend `AIConciergePanel` already round-trips `structured_results` and rehydrates assistant messages via `fromSearchResult`. We have an existing client-side cache (`concierge_cache::tripId::destination`).
- "Trip context" already exists at `trip_id` granularity. Destination change resets the chat's `localStorage` key, and we tighten that further.
- The user is on a single trip per conversation. Multi-trip conversations are out of scope.

2. Success criteria

A turn is correctly handled when:

1. Pure rerank/filter prompts ("top 3", "best one", "date night", "cheapest", "most upscale", "compare these", "only rooftop ones" if data supports) return cards drawn exclusively from the previous turn's verified card set with zero provider calls, in <300 ms server time.
2. Category-or-location-changing prompts ("more options", "restaurants near the first one", "cocktail bars in Wicker Park", a new cuisine, a new neighborhood) trigger a verified provider search exactly like today.
3. The structured-card schema (`UnifiedRestaurantResult` / `UnifiedAttractionResult` / `UnifiedHotelResult` with type: "verified_place", Google verification, `display.*`, `supportingDetails.*`, `whyPick`) is byte-identical to today's output for any reused card.
4. Add-to-trip and Save-to-Ideas continue to work on rerank/filter results without code changes in those handlers.

- 5. Destination/trip change clears conversational context.
- 6. Rerank turns never invent, paraphrase, or upgrade card identity, address, ratings, or links — only reorder/subset.
- 7. Logs show, for every turn: turn-mode, candidate-pool source, provider-call decision, and rerank rule applied. Test-suite coverage exists for each branch in §6.

3. Recommended architecture

One concept added: "turn-mode" classification on top of the existing typed router.

```
ConciergeSearchRequest -> [Stage A: turn-mode classifier] -> {
  NEW_SEARCH      -> existing router/live_research/provider path (unchanged)
  REFINE_PREVIOUS -> new ContextResolver: rerank/subset/filter previous cards
  ANCHOR_NEW      -> live_research with anchor (location seeded from previous card)
  RESET           -> drop context, treat as NEW_SEARCH
}
```

Stage A runs before the existing route_prompt so we don't disturb the typed-response contract. place_recommendations responses produced by REFINE_PREVIOUS are constructed directly from the previous response's verified cards, run through a small deterministic rerank/filter, and re-serialized through the same ConciergeSearchResponse Pydantic model that today's path emits. Same display.*, same whyPick, same identity fields. Frontend renders identically.

Why this shape:

- Surgical: no schema changes, no new table, no router rewrite. One new module (app/concierge/context.py) plus a small classifier.
- Trust-preserving: refine path can only emit cards that came from a previous verified provider response. It cannot introduce new names.
- Backwards-compatible: when classifier says NEW_SEARCH, behavior is unchanged.

4. v1 storage decision: backend-aware, DB-backed via existing concierge_messages

Option	Decision
Frontend-state only	Rejected. Refresh, multi-device, and audit log all break. Trust evaluation should happen server-side because frontend mock-quarantine logic already proves trust gates can drift.

Option	Decision
New conversation_context table	Rejected for v1. Premature. The previous turn's structured cards already live in concierge_messages.structured_results. Adding a table now would be the over-generalization principle #2 forbids.
Reuse concierge_messages rows for context	Selected. The most recent assistant row for trip_id whose structured_results.intent is a place-producing intent is the candidate pool.

Add a small derived in-memory "context window" object built per request from the last N (≤ 6) message rows. No new DDL. A future v2 can promote this to a conversation_context row if we ever need cross-session summarization, but v1 doesn't.

Supabase SQL: No for v1.

5. Backend request/response contract

Request — additive, optional fields only

POST /ai/concierge/search (existing endpoint):

```
{
  trip_id: UUID,
  user_query: string,
  client_message_id?: string,
  // NEW (optional, additive)
  conversation: {
    last_assistant_message_id?: UUID,    // hint for context anchor; server still authoritative
    destination_signature?: string      // hash of (trip_id + destination) – frontend-detected reset
  }
}
```

Existing clients without conversation continue to work — server falls back to "load latest assistant message for trip_id."

Response — additive, optional fields only

ConciergeSearchResponse adds three optional fields:

```
turn_mode: "new_search" | "refine_previous" | "anchor_new" | "reset"    # observability + UI hint
context_used: {
  source_message_id?: UUID,
  card_pool_size?: int,
```

```

    filter_applied?: string,          # human-readable, e.g. "rooftop"
    rerank_rule?: string,             # e.g. "date_night" | "top_n" | "cheapest"
    provider_call: bool               # true iff a fresh provider search ran
  }
  warnings: [...]                   # existing; add "filter_unsupported_fell_back_to_search" etc.

```

response_type and the existing place/advice/unsupported discriminator stay unchanged. All card fields are unchanged. No new top-level type.

6. Session/context data shape (in-memory, per request)

```

ContextWindow {
  trip_id
  destination
  # Selected from concierge_messages
  last_assistant: {
    message_id, created_at, intent,
    cards: [UnifiedRestaurantResult | UnifiedAttractionResult | UnifiedHotelResult],
    category_hint: "restaurants" | "bars" | "attractions" | "hotels" | "mixed",
    location_hint: optional(neighborhood/anchor)
  }
  prior_turns: [last 5 user prompts, normalized] # for rerank-rule disambiguation only
  destination_signature: hash(trip_id + lower(destination.trim()))
  reset_reason: optional("destination_change" | "explicit_reset" | "no_prior_cards")
}

```

Built fresh per request from concierge_messages. Capped at 6 most recent rows. No global cache, no new table, no streaming requirement.

7. Prompt / follow-up classification modes

Stage A classifier (deterministic regex + small allow-list, no LLM for v1):

Mode	Trigger	Example
NEW_SEARCH	Names a category and/or location not present in previous turn; or no previous cards exist	"cocktail bars", "ramen in Shibuya", "things to do tomorrow"

Mode	Trigger	Example
REFINE_PREVIOUS	Pure ordinal/superlative/filter referring to "these / them / the list / above" or generic superlatives without category/location: "top 3", "best one", "best for date night", "cheapest", "most upscale", "compare these", "show me 5", "which is closest to my hotel"	"top 3", "best for date night"
REFINE_FILTER (subtype of REFINE_PREVIOUS, falls through to NEW_SEARCH if pool is too small)	Adds a constraint that previous cards may or may not have data for: "only rooftop ones", "vegan only", "open late"	"only rooftop ones"
ANCHOR_NEW	New search anchored to a previous card or area: "near the first one", "around <prev card name>", "same area as #2"	"restaurants near the first one"
RESET	Destination change detected, explicit "start over"/"new chat", or concierge_messages empty	n/a

Each mode carries a rerank_rule enum the resolver consumes. Rules are deterministic for v1:

- top_n(N) — keep first N after applying the active rerank rule (default = previous order from provider/scorer).
- best_one — top_n(1).
- date_night — boost by tags/category matching: cocktail bars, intimate restaurants, romantic neighborhoods; uses existing tags, best_for_tags, cuisine.
- cheapest — sort by price_per_night (hotels) or by Google priceLevel mapping when present (restaurants/bars). If price unknown for >50% of pool, fall through to NEW_SEARCH.
- most_upscale — inverse of cheapest.
- compare — emit existing comparison rendering using the previous cards as the comparison set (no provider call).

Rules sit in a tiny module so adding more is mechanical.

8. Provider-call decision tree

1. Build ContextWindow.
2. If reset_reason set -> NEW_SEARCH.

3. Else classifier returns mode + rule.
4. If mode == NEW_SEARCH or ANCHOR_NEW:
 - run existing live_research/provider path (unchanged).
 - Provider call = YES.
5. If mode == REFINE_PREVIOUS:
 - a. pool = last_assistant.cards
 - b. if pool size == 0 -> NEW_SEARCH.
 - c. apply rerank_rule deterministically, in-memory.
 - d. if rule needs data the pool doesn't have (e.g. "rooftop" with no signal in pool) -> NEW_SEARCH.
 - e. else: emit response with same cards (subset/reorder), provider_call = NO, retrieval_used = NO.
6. ANCHOR_NEW:
 - extract anchor from "near/around/same area as <ref>", resolve to previous card's neighborhood.

Hard rule: no provider call is allowed when classifier returned REFINE_PREVIOUS and the pool has any usable data. This is the loop-prevention guarantee.

Rate guard: the existing per-endpoint cost guardrail in app/core/cost_guardrails.py still applies; refine turns count against a separate "no-provider" key so they don't exhaust the search budget.

9. How previous verified cards are reused safely

Reuse rules — every one of these is a hard invariant the resolver enforces and tests assert:

1. The reused card object is the same Pydantic model instance shape as today (UnifiedRestaurantResult, etc.). It is not rebuilt from raw text or LLM output.
2. No mutation of identity, location, rating, links, or google_verification. The resolver may only:
 - drop a card,
 - reorder the list,
 - in top_n(N), slice the list.
3. No textual paraphrase of whyPick, display.display_why, or supportingDetails.whyPick. We may prepend a short concierge contextual note to the assistant's response summary text only (e.g., "Top 3 from your earlier picks..."); we do not rewrite per-card fields.
4. The existing trust gate (provider_place_id + google_maps_uri + business_status == OPERATIONAL) is re-checked at the boundary; a card that has gone stale is dropped, never patched.
5. If filter/rerank yields zero cards, response falls back to NEW_SEARCH with a warnings entry explaining the fall-through. Never return an empty addable list while pretending the refine succeeded.
6. The reused cards' source label remains whatever the original turn produced (live_search, confirmed_michelin, etc.). We do not re-stamp live_search on refines.
7. cached: false, turn_mode: "refine_previous", provider_call: false together are the single observable signal that this came from context.

10. How to preserve current card output

- ConciergeSearchResponse shape unchanged (only optional fields added).
- Frontend fromSearchResult mapper unchanged for v1; new optional fields can be ignored. (One small change later in v1 PR 3 to surface a tiny "From your earlier list" tag — not a redesign.)
- Add-to-Day and Save-to-Ideas already operate on the structured card object; reused cards plug in identically.
- All existing display invariants (display.displayWhy ↔ supportingDetails.whyPick ↔ top-level whyPick) are preserved because the cards are not modified.
- localStorage cache version (CONCIERGE_CACHE_VERSION = 4) bumped to 5 only if message shape changes for client-side hydration; otherwise leave alone.

11. Logging / observability

Every turn emits one structured log line plus existing concierge.router.* lines:

```
concierge.context.turn
  trip_id, request_id, turn_mode, rerank_rule, filter,
  pool_size_before, pool_size_after,
  provider_call: bool,
  source_message_id, fall_through_reason?
```

Counters (Prometheus or whatever the repo already uses — confirm in PR 1):

- concierge_turn_mode_total{mode=...}
- concierge_provider_call_skipped_total{reason="refine_previous"}
- concierge_refine_fallthrough_total{reason=...}

Existing persist_concierge_request_log gets two new columns added to the JSON metadata: turn_mode, provider_call. No DDL; that table already stores arbitrary JSON.

12. Test plan

Unit (backend):

- Classifier table-driven tests for ~30 prompts covering every example in the brief plus adversarial: "the top 3 things to do" (ambiguous -> NEW_SEARCH), "top 3" with no prior cards (-> NEW_SEARCH), "more options" (-> NEW_SEARCH).
- Resolver tests for each rerank rule: top_n, best_one, date_night, cheapest, most_upscale, compare.
- Filter fall-through test: "only rooftop ones" with pool that has no rooftop signal -> NEW_SEARCH with appended constraint.
- Identity-preservation test: deep-equal previous card vs. reused card minus list position.
- Trust gate test: stale card (no provider_place_id) is dropped, not patched.

- No-provider guarantee: assert LiveResearchService.fetch not called when turn_mode == "refine_previous".

Integration (backend):

- Sequence test: "cocktail bars" -> "top 3" -> "best for date night" -> "more options". Assert provider called turns 1 and 4 only.
- Destination change test: trip's destination edited -> next turn classified RESET.
- Concurrent turns: two requests racing for same trip — last assistant message resolution is deterministic by created_at desc.

Frontend:

- Existing AIConciergePanel snapshot test still passes — proves card schema unchanged.
- New: response with turn_mode: "refine_previous" renders the same cards with no extra network call (mock callConciergeSearch).
- localStorage destination-change reset still fires.

Manual wife-test script (4 prompts) documented in HANDOFF.

13. Rollout — split into 4 small PRs

1. PR 1 — Context window read path + classifier (backend only, dark).
 - Add app/concierge/context.py (ContextWindow builder, classifier, decision tree skeleton).
 - Wire it into concierge_search endpoint to only log turn_mode for every request. No behavior change.
 - Add observability fields.
 - Tests: classifier table tests.
 - Files: 2 primary (router/contracts unchanged), ~6 test files. Supabase SQL: No.
2. PR 2 — REFINES_PREVIOUS path (backend, behind feature flag).
 - Add resolver with top_n, best_one, compare rules first (highest signal in wife-testing).
 - Add concierge_context_v1_enabled setting flag, default off.
 - Wire response fields turn_mode, context_used.
 - Tests: identity preservation, no-provider guarantee, zero-pool fall-through.
3. PR 3 — REFINES_FILTER + ANCHOR_NEW + remaining rules (date_night, cheapest, most_upscale, "near/around" anchor).
 - Frontend: tiny "From your earlier list" tag when turn_mode === "refine_previous". No redesign.

- Tests: full sequence integration test.

4. PR 4 — Flag flip + rollout.

- Set flag default-on; document toggle in HANDOFF.md.
- Add the metrics dashboards/alerts.
- Manual wife-test script run captured in HANDOFF.

If PR 2 and PR 3 fit easily under the implementation skill's 6-file ceiling, PR 3 may absorb the frontend tag instead of waiting.

14. Risks and mitigations

Risk	Mitigation
Classifier mis-routes ambiguous prompt ("top 3 hidden gems") and skips a needed search	Default ambiguous -> NEW_SEARCH. Classifier is deny-by-default for refine.
Filter rule says "rooftop" but pool has no rooftop signal -> user gets nothing	Hard fall-through to NEW_SEARCH with appended constraint, never empty.
Stale concierge_messages row used as pool after destination change	destination_signature mismatch -> RESET; also re-validate trust gate per card.
Two concurrent requests on same trip race the context	Resolution is order by created_at desc limit 1; the loser refines an older pool — acceptable.
Frontend localStorage cache rehydrates a refine response across destinations	Cache key already includes destination; we keep that.
LLM-rewritten whyPick drift creeps in	Resolver explicitly forbidden from touching display.*/whyPick/supportingDetails.*; tested.
Cost: refine turns still hit Claude for the summary response string	Acceptable; summary is short, no provider/Google call. Optionally skip Claude on top_n/compare and synthesize a one-liner deterministically — propose in PR 2.

15. Non-goals

- No multi-turn LLM tool-use loop. Classifier stays deterministic in v1.
- No new DB table, no schema change, no migration.
- No UI redesign; one small text tag at most.
- No cross-trip memory.
- No personalization / user preference learning.

- No conversational mode for trip_advice or unsupported types — v1 covers place_recommendations only.
 - No streaming responses.
 - No removal of, or weakening of, the Google verification trust gate or the mock-quarantine guards.
-

Final classification

- Severity classification: Level 3
- Root cause/plan: Stateless concierge => every turn re-fetches. Add deterministic turn-mode classifier + reuse-only resolver on top of existing typed router; reuse concierge_messages rows as the context pool; no schema changes. Split into 4 small PRs.
- Files changed: None (spec only).
- Tests: Defined in section 12; not implemented.
- Risks: section 14.
- Supabase SQL: No (v1 deliberately avoids DDL).
- HANDOFF.md edited: No + reason: spec-only pass per request, no implementation.
- README.md edited: No + reason: no public/setup change.